

Reviewer Pack — Minimal Order of Integer Partitions and Resolution Proof Width...

Entry 78145cdcebd8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 06:00:28 UTC

Minimal Order of Integer Partitions and Resolution Proof Width Inequality

Entry ID: 78145cdcebd8 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-07 06:00:28 UTC

1. Conjecture

Field A (mathematical branch): Integer Partition Theory **Field B** (complexity object): Boolean Satisfiability (Resolution Proof Complexity)

Statement:

For every Boolean satisfiability instance φ with n variables, the minimal order of integer partitions $p(\varphi)$ associated with φ satisfies $|p(\varphi)| = O(n^{1/4 + \varepsilon})$ for some constant $\varepsilon > 0$.

Rationale (proposer's reasoning):

Integer partition theory provides a rich structure for encoding various combinatorial objects. The conjecture aims to explore a novel connection between the complexity of Boolean satisfiability and the properties of integer partitions, potentially revealing deep structural insights into resolution proof complexity.

Taxonomy category: integer_partition_theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 46484edef46ae15c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for 30 random CNF formulas, the mean minimal order of integer partitions $|p(\varphi)|$ exceeds $n^{1/4 + \varepsilon}$ by at least 5 standard deviations with a p-value less than 0.05.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 1 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal order of integer partitions" AND "Boolean satisfiability" AND resolution proof complexity" - "integer partition theory" AND "resolution proof width inequality" - "Boolean satisfiability problem" AND "partition functions"

Top relevant hits considered: - [<http://arxiv.org/abs/1310.2252v1>] Kramers-Wannier Duality of Statistical Mechanics Applied to the Boolean Satisfiability Problem of Computer Science

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        variables = list(range(1, n + 1))
        clauses = []
        for _ in range(n):
            clause = random.sample(variables, random.randint(1, n))
            clauses.append(clause)
        return clauses

    def min_order_of_partitions(clauses):
        # Simplified dynamic programming approach to find the minimal order of partitions
        n = len(clauses)
        dp = [[0] * (n + 1) for _ in range(n + 1)]
        for i in range(1, n + 1):
            dp[i][i] = 1
        for length in range(2, n + 1):
            for start in range(1, n - length + 2):
                end = start + length - 1
                dp[start][end] = min(dp[start][k] + dp[k + 1][end] for k in range(start, end))
        return dp[1][n]

    def calculate_metric(n, p):
        return abs(p - n ** (0.25 + 0.01)) # Using a small ε to ensure non-triviality

    results = []
    for n in [5, 10, 15, 20, 30, 40]:
```

```

    p = min_order_of_partitions(generate_cnf(n))
    metric_value = calculate_metric(n, p)
    results.append(metric_value)

mean_value = sum(results) / len(results)
std_dev = math.sqrt(sum((x - mean_value) ** 2 for x in results) / len(results))

conjecture_holds = all(x > n ** (0.25 + 0.01) for x, n in zip(results, [5, 10, 15, 20, 30, 40]))
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "min_order_of_partitions",
    "metric_value": mean_value,
    "instances_tested": len(results),
    "n_max": max([5, 10, 15, 20, 30, 40]),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(x["metric_value"] for x in results) / len(results)
    std_dev = math.sqrt(sum((x["metric_value"] - mean_value) ** 2 for x in results) / len(results))
    support_fraction = sum(1 for x in results if x["conjecture_holds"]) / len(results)

    if all(x["conjecture_holds"] for x in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(x["seed"] for x in results if not x["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

_max': 40, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'min_order_of_partitions', 'metric_value': 17.904823002884452, 'instances_tested': 6}
TRIAL: {'metric_name': 'min_order_of_partitions', 'metric_value': 17.904823002884452, 'instances_tested': 6}
TRIAL: {'metric_name': 'min_order_of_partitions', 'metric_value': 17.904823002884452, 'instances_tested': 6}
TRIAL: {'metric_name': 'min_order_of_partitions', 'metric_value': 17.904823002884452, 'instances_tested': 6}
TRIAL: {'metric_name': 'min_order_of_partitions', 'metric_value': 17.904823002884452, 'instances_tested': 6}
TRIAL: {'metric_name': 'min_order_of_partitions', 'metric_value': 17.904823002884452, 'instances_tested': 6}
TRIAL: {'metric_name': 'min_order_of_partitions', 'metric_value': 17.904823002884452, 'instances_tested': 6}
TRIAL: {'metric_name': 'min_order_of_partitions', 'metric_value': 17.904823002884452, 'instances_tested': 6}
TRIAL: {'metric_name': 'min_order_of_partitions', 'metric_value': 17.904823002884452, 'instances_tested': 6}
TRIAL: {'metric_name': 'min_order_of_partitions', 'metric_value': 17.904823002884452, 'instances_tested': 6}

```

TRIAL: {'metric_name': 'min_order_of_partitions', 'metric_value': 17.904823002884452, 'instances_tested': 40}
 TRIAL: {'metric_name': 'min_order_of_partitions', 'metric_value': 17.904823002884452, 'instances_tested': 40}
 RESULT: SUPPORTED mean=17.904823002884452 std=0.0 support_fraction=1.0

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test only considers n up to 40, which is too small to draw a definitive conclusion about the conjecture's validity for all Boolean satisfiability instances. The metric calculation might not scale trivially with n.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge | original judge: The test results indicate that for all trials up to n=40, the conjecture holds true with a mean value of 17.904823002884452 and standard deviation of | next: Further investigation is needed to confirm the conjecture's validity for all Boolean satisfiability instances beyond n=40. Consider increasing the range of n values in future tests.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14269
2	preregistration	ollama_remote	glm4:latest	0	0	9777
3	novelty	ollama_remote	glm4:latest	0	0	8357
4	novelty	ollama_remote	glm4:latest	0	0	11660
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14013
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8831
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12657
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9920
9	critic	ollama_remote	glm4:latest	0	0	18515
10	judge	ollama_remote	glm4:latest	0	0	15406

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 123405 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/78145cdcebd8.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/78145cdcebd8.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/78145cdcebd8.tar.gz (if generated)